Code Crawler: System Architecture & Design Specification

Goal: A definitive, LLM-first, team-live-syncing, pre-trained-knowledge-aware semantic indexer for massive embedded projects (RDK-B, prplOS, OpenWrt, Buildroot, Android AOSP, Yocto, Linux kernel). Code Crawler minimizes context-window taxation and acts as an intelligence layer with zero retrieval overhead for AI agents.

---

1. System Vision & Philosophy

Code Crawler solves the hardest problems in embedded software analysis: codebase size (often millions of lines), build-system complexity (`#ifdef` hell, Bitbake recipes), and the gap between static code and runtime execution (IPC, hardware logs).

1.1 Core Principles

1. **Agent-Centric Retrieval**: Every LLM interaction with the index must cost ≤ 1 tool call or leverage pre-materialized context bundles. No multi-hop cognitive tax for agents.
2. **Deterministic Graph + Probabilistic Intelligence**: The knowledge graph is 100% deterministic (AST + build configs), while LLMs handle classification, summarization, and proactive patch generation.
3. **Context Window Optimization**: Token processing is expensive. By categorizing code into strict tiers and priority scores, Code Crawler ensures 95% token reduction for agents without losing conceptual accuracy.
4. **Team as a Compute Swarm**: Indexing a 5-million line codebase avoids redundancy. It leverages the team's local AI compute as a distributed swarm, pooling intelligent summaries into a compound team memory.

1.2 The Context Window Problem

The single biggest constraint on LLM-assisted coding is context window utilisation:

Problem	Impact	Code Crawler Solution
"Lost in the middle"	Models perform worst on info buried in the middle of contexts	Tiered indexing ensures only high-priority code enters context
Quadratic attention cost	Token processing is roughly $O(n^2)$; 90% reduction ≈ 99% compute reduction	IndexManifests compress files from ~15K tokens to ~500 tokens
Cost explosion	Scanning a full embedded tree per query is economically insane	Priority scoring ensures only relevant code is retrieved
Multi-hop failures	Intermediary tool calls are failure points for LLM agents	Pre-materialised views eliminate multi-step orchestration

---

2. High-Level Architecture

```
code-crawler/
├── core/                   # Python orchestrator (main engine)
│   ├── pipeline.py         # Indexing pipeline coordinator
│   ├── file_selector.py    # Build-aware file filtering
│   ├── swarm_sync.py       # Distributed Swarm Indexing (P2P workload
sharing)
│   └── hasher.py           # Content hashing for incremental updates
├── crawlers/
│   ├── c/                  # C/C++ (Tree-sitter + libclang hybrid)
│   ├── dts/                # Device Tree Source parser
│   ├── python/             # Python parser
│   ├── shell/              # Shell script parser
│   └── ipc/                # Cross-process definition parser (D-Bus, AIDL,
Ubus)
├── analyzers/
│   └── build/              # Yocto, Buildroot, Linux Kernel parsers
│       ├── detector.py     # Auto-detect build system type
│       ├── yocto.py        # Parse recipes, layers, DISTRO_FEATURES
│       ├── buildroot.py    # Parse .config, package selections
│       ├── kernel.py       # Parse Kconfig, generate compile_commands.json
│       └── compile_db.py   # Unified compile_commands.json handler
├── tiering/                # Tiering & Priority
│   ├── llm_proposer.py     # Sub-7B model directory classifier (i0-i3)
│   ├── priority_scorer.py # 6-dimension self-tuning priority Math
│   └── manifest_builder.py# Pre-calculates 1-call IndexManifests for
agents
├── storage/                # DuckDB multi-model backend
│   ├── schema.py           # Relational + Graph (DuckPGQ) + Vector (vss)
│   ├── git_graphs.py       # Git-aware branch sub-graph management
│   └── vector.py           # Vector index configurations
├── intelligence/           # Background AI and inference
│   ├── proactive_agent.py # Async fix/patch generation on contested
variables
│   ├── summarizer.py       # Confidence-aware tiered summarization
│   └── telemetry.py        # Serial log, dmesg, and crash dump correlator
├── debugger/               # Runtime data integration
│   ├── trace_parser.py     # Parse GDB traces, stack frames
│   ├── sanitizer_parser.py # Parse ASan/Valgrind output
│   └── runtime_scorer.py   # Compute runtime frequency scores
├── collaboration/          # Team live-sync
│   ├── master_sync.py      # Delta sync to shared master DB
│   └── git_patcher.py      # Semantic git patch → graph update
├── mcp/                    # Official Model Context Protocol Server
├── ui/                     # Code Nebula (FastAPI + Three.js 3D Web UI)
├── config/                 # .codecrawler.toml settings
└── plugins/                # Drop-in Rust/C++ extensions for speed
```

---

3. Storage & Data Model

DuckDB with `DuckPGQ` (property graph) and `vss` (vector similarity) provides relational queries, graph traversal, full-text search, and semantic similarity in a single embedded file (eliminating separate vector/graph database deployments).

The database fundamentally physically mirrors the source code directory structure, mapping AI reasoning directly to spatial/file locations.

3.1 Unified Schema

```
-- Core Structural Tables
CREATE TABLE Directory (id BIGINT PRIMARY KEY, path TEXT UNIQUE, name
TEXT, summary TEXT, depth INT, is_custom BOOL);
CREATE TABLE File (id BIGINT PRIMARY KEY, path TEXT UNIQUE, hash TEXT,
last_modified TIMESTAMP, is_custom BOOL, language TEXT, loc INT, embedding
FLOAT[384]);
CREATE TABLE Function (id BIGINT PRIMARY KEY, file_id BIGINT REFERENCES
File(id), name TEXT, signature TEXT, start_line INT, end_line INT, summary
TEXT, complexity INT, embedding FLOAT[384]);
CREATE TABLE Struct (id BIGINT PRIMARY KEY, file_id BIGINT REFERENCES
File(id), name TEXT, summary TEXT, members TEXT[]);
CREATE TABLE Macro (id BIGINT PRIMARY KEY, file_id BIGINT REFERENCES
File(id), name TEXT, value TEXT, is_config_guard BOOL);
CREATE TABLE BuildConfig (id BIGINT PRIMARY KEY, key TEXT, value TEXT,
source_file TEXT, build_system TEXT);
CREATE TABLE DeviceTreeNode (id BIGINT PRIMARY KEY, path TEXT, compatible
TEXT[], properties JSONB, source_file TEXT);

-- Edge Tables
CREATE TABLE contains_dir (parent_id BIGINT, child_id BIGINT);
CREATE TABLE contains_file (dir_id BIGINT, file_id BIGINT);
CREATE TABLE contains_func (file_id BIGINT, func_id BIGINT);
CREATE TABLE calls (caller_id BIGINT, callee_id BIGINT, call_site_line
INT);
CREATE TABLE uses_struct (func_id BIGINT, struct_id BIGINT);
CREATE TABLE includes_file (source_id BIGINT, target_id BIGINT);
CREATE TABLE guarded_by (func_id BIGINT, config_id BIGINT);
CREATE TABLE dt_binds_driver (dt_node_id BIGINT, func_id BIGINT);

-- Cross-Boundary & Telemetry Tables
CREATE TABLE calls_over_ipc (caller_func_id BIGINT, callee_func_id BIGINT,
interface_name TEXT);
CREATE TABLE LogLiteral (id BIGINT PRIMARY KEY, hash TEXT, literal_string
TEXT, log_level TEXT);
CREATE TABLE emits_log (func_id BIGINT REFERENCES Function(id), log_id
BIGINT REFERENCES LogLiteral(id));

-- Tiering & Intelligence Tracking
CREATE TABLE Tier (id BIGINT PRIMARY KEY, path TEXT UNIQUE, tier INT CHECK
(tier BETWEEN 0 AND 3), source TEXT, confidence FLOAT, last_classified
TIMESTAMP);
CREATE TABLE Variable (id BIGINT PRIMARY KEY, func_id BIGINT REFERENCES
Function(id), file_id BIGINT REFERENCES File(id), name TEXT, var_type
```

```
TEXT, is_global BOOL, is_static BOOL, usage_count INT, write_count INT,
priority_score FLOAT, embedding FLOAT[384]);
CREATE TABLE IndexManifest (file_id BIGINT PRIMARY KEY REFERENCES
File(id), manifest_json JSONB);
CREATE TABLE PriorityScore (func_id BIGINT PRIMARY KEY REFERENCES
Function(id), tier_weight FLOAT, usage_frequency FLOAT, graph_centrality
FLOAT, build_guard_activation FLOAT, runtime_frequency FLOAT,
recency_score FLOAT, composite_score FLOAT);
CREATE TABLE SummaryMeta (entity_id BIGINT, entity_type TEXT, model_used
TEXT, confidence FLOAT, version INT, created_at TIMESTAMP, PRIMARY KEY
(entity_id, entity_type));

-- Runtime & Debug Data
CREATE TABLE RuntimeTrace (id BIGINT PRIMARY KEY, func_id BIGINT
REFERENCES Function(id), source TEXT, hit_count INT, avg_stack_depth
FLOAT, has_memory_error BOOL, last_seen TIMESTAMP, trace_data JSONB);

-- Collaboration & Feedback
CREATE TABLE SyncLog (id BIGINT PRIMARY KEY, entity_id BIGINT, entity_type
TEXT, change_type TEXT, changed_by TEXT, commit_sha TEXT, timestamp
TIMESTAMP, delta_json JSONB);
CREATE TABLE Annotation (id BIGINT PRIMARY KEY, entity_id BIGINT,
entity_type TEXT, annotation_type TEXT, content TEXT, model TEXT,
confidence FLOAT, approved BOOL DEFAULT FALSE, created_at TIMESTAMP);
```

3.2 Pre-Materialized LLM Views

```
-- One-call view: all high-priority functions in a layer
CREATE VIEW LLM_HighPriority AS
  SELECT f.*, ps.composite_score, t.tier, sm.confidence as
summary_confidence
  FROM Function f JOIN PriorityScore ps ON f.id = ps.func_id JOIN
contains_func cf ON f.id = cf.func_id JOIN contains_file cfl ON cf.file_id
= cfl.file_id JOIN Tier t ON t.path = (SELECT path FROM File WHERE id =
cf.file_id)
  WHERE t.tier >= 2 ORDER BY ps.composite_score DESC;

-- One-call view: shared state flag for proactive remediation
CREATE VIEW LLM_SharedState AS
  SELECT v.*, f.name as func_name, fi.path as file_path
  FROM Variable v JOIN Function f ON v.func_id = f.id JOIN File fi ON
v.file_id = fi.id
  WHERE v.is_global = TRUE AND v.write_count > 1 ORDER BY v.write_count
DESC;

-- One-call view: functions flagged by debugger/runtime data
CREATE VIEW LLM_RuntimeHotspots AS
  SELECT f.*, rt.hit_count, rt.has_memory_error, rt.source as
trace_source, ps.composite_score
  FROM Function f JOIN RuntimeTrace rt ON f.id = rt.func_id LEFT JOIN
```

```
  PriorityScore ps ON f.id = ps.func_id
    ORDER BY rt.hit_count DESC;
```

3.3 Graph Definition (DuckPGQ) & Vectors

```
CREATE PROPERTY GRAPH code_graph
  VERTEX TABLES (Directory, File, Function, Struct, Macro, BuildConfig,
DeviceTreeNode, Variable, LogLiteral)
  EDGE TABLES (
    contains_dir SOURCE KEY (parent_id) REFERENCES Directory DESTINATION
KEY (child_id) REFERENCES Directory,
    contains_file SOURCE KEY (dir_id) REFERENCES Directory DESTINATION KEY
(file_id) REFERENCES File,
    contains_func SOURCE KEY (file_id) REFERENCES File DESTINATION KEY
(func_id) REFERENCES Function,
    calls SOURCE KEY (caller_id) REFERENCES Function DESTINATION KEY
(callee_id) REFERENCES Function,
    uses_struct SOURCE KEY (func_id) REFERENCES Function DESTINATION KEY
(struct_id) REFERENCES Struct,
    includes_file SOURCE KEY (source_id) REFERENCES File DESTINATION KEY
(target_id) REFERENCES File,
    guarded_by SOURCE KEY (func_id) REFERENCES Function DESTINATION KEY
(config_id) REFERENCES BuildConfig,
    dt_binds_driver SOURCE KEY (dt_node_id) REFERENCES DeviceTreeNode
DESTINATION KEY (func_id) REFERENCES Function,
    calls_over_ipc SOURCE KEY (caller_func_id) REFERENCES Function
DESTINATION KEY (callee_func_id) REFERENCES Function,
    emits_log SOURCE KEY (func_id) REFERENCES Function DESTINATION KEY
(log_id) REFERENCES LogLiteral
  );

INSTALL vss; LOAD vss;
CREATE INDEX func_embedding_idx ON Function USING HNSW (embedding) WITH
(metric = 'cosine');
CREATE INDEX file_embedding_idx ON File USING HNSW (embedding) WITH
(metric = 'cosine');
CREATE INDEX var_embedding_idx ON Variable USING HNSW (embedding) WITH
(metric = 'cosine');
```

---

4. LLM-Assisted Tiering & Classification

4.1 The Four Index Tiers

Tier	Label	What Gets Indexed	Examples
i0	Ignore	Nothing. Completely skipped.	binutils, gcc, glibc, busybox, toolchain, kernel upstream untouched subsystems
i1	Stub	File name + path + hash only	System libs, coreutils, generic drivers

Tier	Label	What Gets Indexed	Examples
i2	Skeleton	Signatures + call edges + struct defs (no summaries)	Integration layers, APIs, OpenWrt feeds, Android HAL stubs
i3	Full	Complete AST + summaries + vectors + variable tracking	Custom vendor layers, developer HAL, prplOS TR-181, app code

4.2 Two-Phase Classification Pipeline

Code Crawler avoids brittle regex by utilizing the **pre-trained knowledge** of local LLMs. Models trained on giant codebases instinctively recognize the difference between Linux kernel internals and vendor-custom drivers.

```
Phase 1: Pre-Trained Knowledge Classification
──────────────────────────────────────────────────────
Directory tree + build metadata (Yocto layers, .config, IMAGE_INSTALL, DT
files)
    ▼
Cheap local LLM (Llama-3.2-3B / Qwen2.5-3B)
    ▼
Initial i0–i3 proposal for every top-level directory

Phase 2: Git Evidence Validation
──────────────────────────────────
For each directory classified as i0 or i1:
    git log --oneline --since="6 months ago" -- <dir>
    ├── Has recent commits? → Bump to i2 minimum
    └── No recent commits? → Keep classification

Phase 3: Build-Config Cross-Reference
──────────────────────────────────────────────
Cross-reference with enabled packages, active configs, IMAGE_INSTALL
    ├── Directory contains active build target? → Bump to i2+
    └── Directory excluded from build? → Keep i0/i1
```

---

5. Priority Scoring System (Mathematical Framework)

Every indexed function evaluates a Priority Score, eliminating bloated contexts.

5.1 The Composite Score Formula

```
composite_score = (tier_weight    × W_t) +
                  (usage_freq     × W_u) +
                  (centrality     × W_c) +
                  (build_active   × W_b) +
                  (runtime_freq   × W_r) +
                  (recency        × W_e)
```

Dimension	Default Weight	Formula / Source	What It Tells the LLM
Tier	`W_t = 0.25`	`tier_level / 3.0`	"This is code the developer is actively working on."
Usage	`W_u = 0.20`	`call_count(f) / max_counts`	Identify "hub" functions with massive ripple impacts.
Centrality	`W_c = 0.15`	`betweenness_centrality()`	Identifies graph bottlenecks and bridging APIs.
Build Guard	`W_b = 0.10`	1.0 if `#ifdef` matches config	Unused paths are effectively dead tokens.
Runtime	`W_r = 0.15`	From debugger trace hits	Actual runtime vs static expectation.
Recency	`W_e = 0.15`	`1/(1 + days_since_mod)`	Smooth decay exponential backoff for modified code.

Self-Tuning Weights (Meta-Learning): The system continuously adjusts `W_t` through `W_e`. If an engineering team heavily queries functions based on recent commits or crash traces, the system auto-adjusts `W_e` and `W_r` iteratively over weeks via graph query logging.

---

6. Parsing & Cross-Boundary Edge Resolution

6.1 Per-Language Universal Extraction

Every language module output normalises into the same elements: Functions, Variables, Calls, Parameters, Returns, Structs, Macros, DT Bindings.

Variable Tracking: Variables are first-class nodes. `Global` variables track full cross-function `write_count` metrics. Write counts > 1 trigger proactive logic (see Section 8).

6.2 IPC Semantic Edges (The Missing Link)

In embedded Linux systems, business logic jumps processes boundaries. Code Crawler builds native AST edges spanning D-Bus XMLs, Android AIDL, protobufs, and JSON-RPC.

- If a Python app invokes a `Wifi.SetSSID` DBus signal, an edge links that Python function directly to the C++ daemon handler. Zero LLM context drops across process domains.

6.3 Build System & Device Tree Awareness

- **Yocto / Buildroot / Kernel**: Native parsing of `.config`, layer priority, and Kconfig flags constructs a global `#ifdef` symbol table. libclang leverages this to aggressively filter out dead `#else` branches.
- **Hardware Integration**: Extracts `.dtsi compatible` strings and dynamically wires them to `probe()` functions in the AST via `dt_binds_driver`.

---

7. Telemetry & Runtime Data

Static analysis tells you what code *could* do. Runtime data tells you what it *actually does*.

7.1 Serial Log Correlation (Mapping Telemetry)

Embedded developers live in serial output. Code Crawler runs a pass specifically for logging macros (`ALOGE`, `printk`, `RDK_LOG`).

- Literals are stripped and hashed into `LogLiteral` nodes.
- Agents receive raw 50-line crash logs, and the engine hashes the strings to instantaneously teleport to the correct 5 AST contexts.

7.2 Debugger Integration Pipeline

Debugger Signal	How It Improves the Index
Stack traces (GDB)	Hit-count weight modifier applied to hot-path functions.
Breakpoint hits	Alters static assumptions ("A calls B" vs "A calls B 100K times/sec").
Sanitisers (Valgrind/ASan)	Emits automatic `warning` annotations securely attached to `Function` nodes.

8. Proactive AI & Team Live-Sync

8.1 Tiered Summarisation & Confidence

- Small local models pre-process 0.6-confidence, 2-sentence summaries.
- When an agent/developer examines a file, a background thread lazily upgrades it via a larger model (e.g., Claude 3.5), replacing the summary atomically with a 0.9-confidence, high-context explanation.

8.2 Proactive Background Remediation

The engine acts as a background team member. If an active `Variable` achieves a high `write_count` spanning multiple execution contexts (or interrupts) without a `mutex` in the AST, the background daemon proactively generates a git patch to secure the data, surfacing it inside the UI as a `suggested_patch` annotation.

8.3 Collaboration via Swarm Compute

Indexing millions of lines is computationally heavy.

- **Swarm Compute**: When a team updates a large vendor drop, local daemons on the LAN divide the AST/summarisation load (Dev A's GPU processes `fs/`, Dev B processes `net/`), avoiding redundant processing.
- **Master Database Sync**: The resulting metadata, tier upgrades, and verified LLM annotations propagate back to a shared Delta Master DB (`SyncLog`).
- **Git-Aware Sub-Graphs**: Developers working on messy local `feature/*` branches have their new nodes isolated in branch-specific sub-graphs, ensuring experimental annotations don't pollute the

master intelligence engine until a merge commits.

8.4 Git Semantic Patching

On `git pull`, the DB executes incremental updates via file hashing. For deeply understood `i3` files, an LLM compares the git patch against the `IndexManifest` to generate a natural-language semantic graph delta without needing a full-file AST rescrape.

---

9. Main Engine Design

9.1 CLI Interface

```
codecrawler index --project yocto --image rdk-b --config .codecrawler.toml
codecrawler mcp             # starts MCP server
codecrawler ui              # starts Code Nebula 3D dashboard
codecrawler watch           # real-time incremental daemon
codecrawler sync            # Swarm sync with team master DB
codecrawler ingest-logs     # Ingest fleet crash logs / serial traces
codecrawler status          # show index statistics
```

9.2 The Indexing Pipeline

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                      INDEXING SWARM PIPELINE
│
│
│   ┌──────────┐     ┌──────────┐     ┌──────────┐     ┌──────────┐
│   | 1. Quick |──▶| 2. Build  |──▶| 3. LLM     |──▶| 4. Hybrid   |
│   | Dir Scan |     | Detector  |     | Tier Class.|     | Tier Merge  |
│   └──────────┘     └──────────┘     └──────────┘     └──────────┘
│                                                            |
│                                                  ┌──────────┘
│                                                  ▼
│   ┌──────────┐     ┌──────────┐     ┌──────────┐     ┌──────────┐
│   | 8. Graph  |◀──| 7. Fleet  |◀──| 6. LLM     |◀──| 5. Intersect |
│   | Ingest    |     | Telemetry |     | Summarize   |     | IPC + AST    |
│   └──────────┘     └──────────┘     └──────────┘     └──────────┘
```

```
│
│       │
│
│       ▼
│
│  ┌──────────┐    ┌──────────┐    ┌──────────────┐    ┌──────────────┐
│
│  │ 9. Vector │──▶│ 10. Math  │──▶│ 11. Bundle   │──▶│ 12. P2P      │
│
│  │ Embeds    │    │ Scorer    │    │ Manifests    │    │ Swarm Sync   │
│
│  └──────────┘    └──────────┘    └──────────────┘    └──────────────┘
│
└──────────────────────────────────────────────────────────────────────────
┘
```

---

10. MCP (Model Context Protocol) Integration

10.1 Key MCP Tools

Tool	Description	Returns
`search_code(query)`	Vector + FTS unified search	Ranked `IndexManifest` bundles
`get_call_hierarchy`	Graph traversal	Tree with signatures + summaries
`get_build_context`	`#ifdef` config lookup	Active paths affecting the query
`trace_ipc_flow`	Spans DBus/Ubus architectures	Ordered cross-process dependencies
`correlate_serial_log`	Log extraction jump	AST paths emitting exact strings
`analyze_impact`	Dependency blast radius	Downstream hardware/software impacts
`sync_team()`	Distributed swarm poll	Applies remote team summaries/patches

10.2 Pre-Materialised MCP Resources

Resource URI	Description
`codecrawler://manifest/{path}`	Complete per-file context bundle (~500 tokens)
`codecrawler://llm_view/{layer}`	Pre-built high-priority SQL views for immediate context
`codecrawler://telemetry/{node}`	Recent crashes / warnings for this specific subsystem

---

11. UI — Code Nebula

The **FastAPI + Three.js** interactive 3D graph (operable in browser) functions as the "Google Earth" of the codebase.

- **Nebula Spatial Views**: Visualizes files organically. Nodes sized by priority, colored by Tier.
- **Telemetry Heatmaps**: Dims everything except functions physically touched by active GDB/Valgrind traces.
- **Tinker Focus**: Darkens untouched Linux/Busybox layers to highlight strictly the vendor apps and IPC bridges.
- **Nebula Tour**: Generates an automated LLM fly-through describing exactly how a subsystem boots and operates structurally.

---

12. Configuration (.codecrawler.toml)

```toml
[project]
name = "my-rdk-build"
type = "yocto"
root = "/home/dev/yocto-build"

[index]
tiers = { full = ["meta-custom/**", "meta-vendor/**"], skeleton =
["poky/meta/**"], stub = ["**"] }

[build]
config_file = "build/conf/local.conf"
layers_file = "build/conf/bblayers.conf"
kernel_config = "build/tmp/work/**/linux-*/build/.config"
compile_commands = "auto"

[llm]
provider = "ollama"
model = "llama3.2:8b"

[embeddings]
model = "sentence-transformers/all-MiniLM-L6-v2"
device = "cpu"

[tiering]
llm_proposer_model = "llama3.2:3b"
git_evidence_months = 6

[priority_scoring]
weights = { tier = 0.25, usage = 0.20, centrality = 0.15, build = 0.10,
runtime = 0.15, recency = 0.15 }
self_tuning = true

[collaboration]
enabled = true
master_db_path = "shared/codecrawler_master.db"
swarm_compute = true
developer_id = "dev-a"
```

```
[git]
semantic_patch_enabled = true
branch_isolation = true

[telemetry]
enabled = true
sources = ["gdb_traces", "valgrind", "asan_logs", "serial_uart_logs"]
auto_patch_generation = true
```

---

13. Error Handling, Security & Resilience

- **Secure Execution**: MCP tools execute locally and return data strictly; no arbitrary LLM execution commands exist in the core DB.
- **Branch Bleed Prevention**: Git-aware sub-graphs completely eliminate unfinished code infecting the master analytics engine.
- **Database Degradation**: If Swarm Sync drops, DuckDB caches DeltaSyncLogs locally and applies when connectivity restores.
- **Parsing Fallbacks**: If `libclang` fails due to broken `compile_commands.json`, Tree-Sitter seamlessly abstracts the structural components without interrupting the UI flow.

---

14. Technology Stack

Component	Technology	Rationale
Core & Engine	Python 3.12+	Rich AST and AI ecosystem bindings.
Parsing Pipeline	`tree-sitter` + libclang	Fast fault-tolerant structures + deep C semantic layers.
Database	DuckDB + DuckPGQ + vss	Zero-admin, unified graph+relational+vector store.
AI Intelligence	Ollama (Llama 3 8B, Qwen)	Local, free, massive pre-trained open source code knowledge.
Embeddings	`sentence-transformers`	Sub-second local semantic generation.
Graph Centrality	`networkx`	Betweenness-centrality for prioritizing bottlenecks.
UI	FastAPI + Three.js	Lightweight spatial mapping without massive React payloads.

---

15. Execution Phases

Phase 0 – Storage & Structural Foundation

- [ ] Project skeleton with DuckDB + PGQ schema definitions.

- ☐ Universal parser integration (C/C++ Tree-sitter + variable/struct scraping).

Phase 1 – Build Intelligence & Boundaries

- ☐ Build detector (Yocto, Buildroot, Kernel config parsing).
- ☐ Device Tree hardware matching.
- ☐ IPC Boundary detection (D-Bus, AIDL bridging schema mappings).

Phase 2 – LLM Tiering & Mathematics

- ☐ Sub-7B tier proposer deployment (i0-i3 classifications).
- ☐ Priority Scoring engine (6-dimension scoring with Recency calculation).
- ☐ Log telemetry hasher/correlator (Macro `printk` extractor).

Phase 3 – MCP & Swarm Synchronization

- ☐ Swarm P2P orchestration pipeline.
- ☐ Git-aware Sub-graphs layout.
- ☐ MCP tool implementations and `IndexManifest` builder routes.

Phase 4 – Proactive Intelligence & Polish

- ☐ Background remediation agent (write-contention patch generator).
- ☐ Three.js "Code Nebula" web UI + Nebula Flythrough Tours.
- ☐ Performance and Success metric verification.

---

16. Success Metrics

Metric	Target Goal
Indexing Speed	> 1,000 files/min (Multiplied dynamically per Swarm peer joining)
Query Latency (MCP)	< 200ms per retrieval
Context Compression	~30:1 token reduction via pre-packaged `IndexManifests`
First Useful AI Query	< 90 seconds (Leveraging LLM tier-skips over indexing delays)
Team Sync Latency	< 2 seconds delta replication
Agent Retrieve Burden	Capped strictly at 1 MCP call or 1 pre-calculated View